

Interfaces

Classes and interfaces

class	Interface
The members of a class can be constant or variables	Members are constant, their values are final
Methods can be abstract or non abstract	Methods can be abstract only.
Can be instantiated	Cannot be instantiated
Various access specifiers like public, private, protected etc can be used.	only public access specifiers

- An **interface** in Java is a reference type that defines a contract for classes to implement.
- It contains **abstract methods** (methods without a body) and **constants** (static final variables).
- Interfaces allow for multiple inheritance in Java and help achieve abstraction.

Key Characteristics of Interfaces

- 1. Method Declaration Only:** Methods in interfaces do not have implementations
- 2. Multiple Inheritance:** A class can implement multiple interfaces, overcoming the limitation of single inheritance.
- 3. Public and Abstract by Default:** All methods in an interface are implicitly public and abstract.
- 4. Constants:** Variables in an interface are implicitly public, static, and final.
- 5. No Constructors:** Interfaces cannot have constructors since they cannot be instantiated.

Step 1: Define the Interface

```
// Interface declaration
interface Vehicle {
    void start(); // Abstract method (implicitly
public and abstract)

    default void stop() {
// Default method (with implementation)
        System.out.println("Vehicle is
stopping...");
    }
}
```

Step 2: Implement the Interface in a class

// Implementing the interface in Car
class

- class Car implements Vehicle {
- @Override
- public void start() {
- System.out.println("Car is
starting...");
- }
- }

Step 3: Another Class Implementing the Same Interface

- `// Implementing the interface in Bike class`
- `class Bike implements Vehicle {`
- `@Override`
- `public void start() {`
- `System.out.println("Bike is starting...");`
- `}`
- `}`

Main method

- public class InterfaceExample {
- public static void main(String[] args) {
- Vehicle myCar = new Car();
- myCar.start(); // Calls start method in Car
- myCar.stop(); // Calls default method from interface
- Vehicle myBike = new Bike();
- myBike.start(); // Calls start method in Bike
- myBike.stop(); // Calls default method from interface
- }
- }

Output

- Car is starting...
- Vehicle is stopping...
- Bike is starting...
- Vehicle is stopping...